Verified Optimizations to Flock's RS Prover

measurements on Apple M1 Max, single-threaded; zerocheck at 2^{18}, PCS open at 2^{16}

BLAKE3 compressions

branch `snark-fast-improvements`

August 2026

Abstract

Seven changes to the RS (additive-NTT) zerocheck path, each kept only after a paired, order-alternating A/B measurement with a decisive sign test. Together they reduce zerocheck round 1 from 1477 ms to 1205 ms (−18.4%), round 2 from 433 to 312 ms (−28%), the rounds-3+ tail from 455 to 307 ms (−33%), and whole-proof time by ≈ 11% single-threaded. Three are ideas ported from an independently optimized fork of this prover (the "challenge" tree), re-derived and re-implemented rather than copied; three originated here. The recurring themes: reductions in fields of characteristic two are $\mathbb{F}_2$-linear and can be deferred almost arbitrarily; work that a circuit fixes structurally can be skipped with a one-word guard; and on Apple silicon the boundary between the vector and general register files costs more than most instruction counts. A later section extends the record to the PCS open, whose dominant pass loses its per-slot field multiply to the same $\mathbb{F}_2$-linearity argument (§6.1).

Preliminaries

Shared definitions and the benchmark shape. The numbered sections group the work by prover phase, in pipeline order; each subsection *is one verified optimization, presented with its mechanism and its paired measurement.*

Fields. $\mathbb{F}_{2^8} = \mathbb{F}_2[x]/(x^8+x^4+x^3+x+1)$ and the GHASH field $\mathbb{F}_{2^{128}} = \mathbb{F}_2[X]/(X^{128}+X^7+X^2+X+1)$, with the ring embedding $\varphi : \mathbb{F}_{2^8} \hookrightarrow \mathbb{F}_{2^{128}}$. Write $\alpha := \varphi(x)$ and $\gamma := X$.

The witness is $z \in \mathbb{F}_2^{2^m}$; at the benchmark shape $m = 32$ (2^{18} BLAKE3 compressions, $k_{\log} = 14$ bits per block). After the univariate skip, round 1 of the zerocheck views the m index bits as

$$[\ \underbrace{\lambda}_{6\ \text{(univariate)}}\ |\ \underbrace{s}_{3\ \text{(small)}}\ |\ \underbrace{b}_{4\ \text{(medium)}}\ |\ \underbrace{t}_{m-13\ \text{(rest)}}\ ],$$

and must emit, for each of the 64 points λ of the skip domain, the quadratic message $P^{AB}(\lambda)$ and the linear message $P^{C}(\lambda)$, plus a pair of *banks* (the ring-switch helper $\hat{s}_{v,C}$) that keep the small parity dimension unfolded.

The pinned values below are protocol design, common to both trees — not one of this document's optimizations. The optimization content is *which direction* a kernel exploits them; the two directions are stated in math at the end of this section. The protocol pins the small and medium challenges to *friendly* values: $r_{6+i} = \varphi(v_i)$ with $v = (\texttt{0xF7}, \texttt{0x53}, \texttt{0xB5})$, and

$\beta_i = \gamma^{2^{i-1}}/(1+\gamma^{2^{i-1}})$. These make the eq weights geometric:

$$\prod_{i=0}^{2} \mathrm{eq}(r_{6+i}, K_i) \;=\; C_s\, \alpha^K, \qquad \prod_{i=1}^{4} \mathrm{eq}(\beta_i, b_i) \;=\; \gamma^b/D, \tag{1}$$

with $K = \sum K_i 2^i$, $C_s = \prod_i (1 + r_{6+i}) = \varphi(\texttt{0x1C})$, and $D = \prod_i (1 + \gamma^{2^{i-1}})$. The baseline kernels exploit (1) through lookup tables: the geometric weights are enumerated once into byte-indexed convert tables and applied by gathers. Three optimizations below exploit it in the opposite direction, replacing tables by folds at the actual challenge values: the C-side banks are recomputed as a true multilinear fold at the pinned challenges, with the over-folded eq factors undone by a closed-form constant that exists only because of (1) (§2.2); the prep kernel's weight x^K is factored so one piece stays a table image while the x^2 piece is applied as a shift-and-conditional-XOR directly on the operand (§2.1); and the PCS open builds its composed fold table by walking the monomials $X^r e$ with 127 exact shift-and-folds instead of 128 general multiplies (§6.1 — resting on $\gamma = X$ being the generator rather than on (1) itself).

The two directions, in math. Both evaluate contractions against the same geometric weight vector: with $w_K = C\,\alpha^K$ for $K < n$ and bit-inputs $u \in \mathbb{F}_2^n$, the quantity is $\langle w, u\rangle = \sum_{K<n} u_K\, w_K$. The *table* direction precomputes the entire linear map once,

$$T[u] \;=\; \sum_{K<n} u_K\, w_K \quad \text{for all } u \in \mathbb{F}_2^n \qquad (2^n \text{ entries, built by subset-sum doubling}),$$

after which each of N streamed inputs costs a single gather, $\langle w, u^{(j)}\rangle = T[u^{(j)}]$: setup $O(2^n)$, then per-input cost independent of n and no multiplies at all. The *fold* direction never materializes T; it applies the weights level by level to the resident data itself,

$$u \;\leftarrow\; u_{\text{even}} \;+\; \alpha^{2^i}\, u_{\text{odd}}, \qquad i = 0, \ldots, \log_2 n - 1,$$

costing $n - 1$ multiplies per resident vector and touching no table memory. Hence the rule separating them: the table amortizes when one fixed map meets a stream of many inputs ($N \gg 2^n/n$, the load-port-bound drains); the fold wins when there is essentially one resident object — an accumulator, a bank, or the table T itself while it is being constructed (subset-sum doubling *is* a fold; §6.1 is that observation at $n = 128$). Folds at freshly *sampled* challenges (rounds two and onward) get no such shortcut — there a fold is an ordinary multiply.

Measurement protocol. All numbers are single-threaded at the shape above. Each verdict is a paired A/B: both arms built once, one discarded warm-up each, then eight pairs with the *order alternating within each pair* (a fixed order is biased by thermal drift by roughly the size of the effects measured here). Phase times are the minimum over the ∼5 zerocheck invocations within one run. A change is kept only on a decisive sign test; two of the six wins were 8/8 with disjoint ranges. Measure only on AC power with the battery above ∼ 60%: low battery caps frequencies, and fast-charging a nearly empty battery is worse.

1 Witness generation

Everything downstream consumes the bit-packed witness; its builder is the first timed phase.

1.1 Streaming full-word publication

Baseline: OR sub-word fields into pre-zeroed storage: a ≈130 MB×3 memset plus a read-modify-write on every store.

Improvement: A sequential writer that carries a partial word in a register publishes complete u64s in order and never reads the destination.

Mechanism: Eliminates the ≈130 MB×3 memset and the per-store read-modify-write entirely.

Result: ST 87–90 → 64.8–68.9 ms (−24%, 3/3 disjoint); 8T −20%.

The BLAKE3 witness builder fills three bit-packed buffers (z, a, b; one bit per R1CS wire) per compression block. The incumbent wrote them by OR-ing sub-word fields into pre-zeroed storage: a ≈130 MB ×3 memset, then a read-modify-write on every store. But the circuit's rows are *contiguous* through the block's useful bits, so a sequential writer that carries a partial word in a register can publish complete u64s in order and never read the destination — the memset and the RMW tax both disappear. The one out-of-order region (the output chaining value, 256-bit aligned) is reserved and overwritten once the final state is known. The fixed 56-G compression schedule is also fully unrolled with literal state/message indices, exposing the whole dependency graph to register allocation. Bit-identical through the driver (tested against the OR builder on real and padding slots, both values of the prefix carry bit). Measured, paired: ST 87–90 → 64.8–68.9 ms (−24%, 3/3 disjoint); 8T −20%. Idea from the challenge tree; their further ≈3–7 ms SIMD-lockstep stage was priced against its ≈400 lines and not taken — the full network was later built anyway and measured null in-prove (§1.2).

1.2 The vectorized-packing port that became an allocation fix

Baseline: The lincheck stripe buffer was a fresh *zeroed* 128–512 MB allocation on every prove; its first-touch page faults dominated the witness bucket.

Improvement: Pool that one buffer (an untyped byte pool alongside the field-element pool) instead of reallocating and zeroing it each prove.

Mechanism: Not an operation-count change but an allocation fix: removes the per-prove first-touch page-fault cost of a fresh 128–512 MB zeroed allocation, at ≈40 lines (versus the ported vectorized-packing network's ≈200 lines, which measured null).

Result: In-prove witness bucket 18.5 → 8.0 ms at $n = 2^{16}$ and 74 → 33 ms at $n = 2^{18}$, both 2/2 paired at 8 threads.

The challenge tree's last witness edge was believed to be its lane-wise vectorized packing network: each bit-packed output stream owns a u32-granular writer whose pending word lives in a NEON lane, every wire bit is pushed by a single `vsli` (shift-left-insert) with compile-time shift constants, four compression blocks proceed in lockstep, and finished words dump contiguously through a `vld4` de-interleave. We reproduced the full design with a `build.rs` generator (≈200 committed lines emitting the unrolled kernel, rather than porting their 2.6k generated lines). It was bit-identical on its first full test run — and measured a *null* in-prove at both shapes.

The discrepancy exposed the real cost, which was never the builder: the lincheck stripe buffer — a repacking of z consumed by the lincheck — was a fresh *zeroed* 128–512 MB allocation on every prove, and its first-touch page faults dominated the bucket. Pooling that one buffer (an untyped byte pool alongside the field-element pool) took the in-prove witness bucket from 18.5 to 8.0 ms at $n = 2^{16}$ and 74 to 33 ms at $n = 2^{18}$, both 2/2 paired at 8 threads — more than the

packing port's entire predicted value, at ≈40 lines. The mechanism is thread-count-agnostic (the buffer is faulted once per prove regardless); the 8-thread figures are simply where the campaign measured it. The quad kernel was reverted unmerged; its design survives in the commit history. This is the third time the campaign's largest available win sat in the allocation story rather than the kernel (the constant-region elision of §8.2, the pooled round-1 precompute, and this): audit where the buffers come from before porting compute.

2 Zerocheck round 1

Round 1 is the zerocheck's largest phase; the four kept changes below took it from 1477 to 1205 ms (−18.4%) at the benchmark shape.

2.1 Unreduced accumulation with multiplicative weight splitting

Baseline: Each y_K computed *reduced*: two `PMULL`s for the raw product plus a reduction costing four more, then a widening shift.

Improvement: Split the weight x^K multiplicatively so the reduction is deferred to the final fold; accumulate unreduced.

Mechanism: 6 `PMULL` → 2 `PMULL` per row-block (Algorithm 1).

Result: Round 1 1401.4 → 1273.4 ms (−9.1%), 8/8, every pair in $[-8.7\%, -10.1\%]$.

The prep kernel computes, per K-row ($K \in \{0, \dots, 7\}$) and 16-lane block, $y_K = a_K \odot b_K \in \mathbb{F}_{2^8}$ and accumulates $\sum_K x^K y_K$ in 16-bit lanes, reducing once at the end. The baseline computed each y_K *reduced*: two `PMULL`s for the raw product plus a reduction costing four more, then a widening shift by K — six `PMULL`s per row-block for a reduction that the final fold makes redundant.

Let u_K be the *unreduced* 15-bit product. Then $\sum_K x^K u_K \equiv \sum_K x^K y_K \pmod{p_8}$, but $x^K u_K$ overflows 16 bits for $K \geq 2$. Split the weight so every factor lands where it is free:

$$x^K = \underbrace{(x^4)^{[K \geq 4]}}_{\text{choice of gather table}} \cdot \underbrace{(x^2)^{[\lfloor K/2 \rfloor \text{ odd}]}}_{\text{byte-wise doubling of the reduced operand}} \cdot \underbrace{x^{K \bmod 2}}_{\text{in-lane shift}}.$$

The x^4 factor costs nothing per element: a second table image with every byte pre-multiplied by x^4 scales each gathered XOR-sum by x^4, by linearity of T. The x^2 factor is a six-instruction shift-and-conditional-XOR on the already-reduced 8-bit operand (deg still ≤ 7). The residual shift is one vector instruction and raises term degree to at most 15.

Algorithm 1 Row-block accumulate, before and after

1: **before:** $acc \oplus= \text{shift}_K\big(\text{reduce}(\text{pmull}(da, db))\big)$ ▷ 6 `PMULL`
2: **after:** $da \leftarrow [K \geq 4]$? gathered from x^4 table : da; $da \leftarrow [(K/2) \bmod 2]$? $x^2 \cdot da$: da
3: $acc \oplus= \text{shift}_{K \bmod 2}\big(\text{pmull}(da, db)\big)$ ▷ 2 `PMULL`

Correctness needs the final reducer to be exact to degree 15, one beyond its documented "≤ 14"; both the scalar and vector reducers were verified exhaustively over all 2^{16} inputs (tests now permanent). Gather traffic is unchanged apart from the second 16 KiB table image.

Measured: round 1 1401.4 → 1273.4 *ms (*−9.1%*), 8/8, every pair in* $[-8.7\%, -10.1\%]$. *The reference implementation's attribution predicted* 100–140 *ms; measured* 128.

2.2 The C-side message as a fold of the lincheck stripe

Baseline: The C-side banks, though a multilinear evaluation, were computed with the quadratic side's machinery: a bit transpose of every 8192-bit window plus 32 of the drain's 48 gathers per lane.

Improvement: Compute the banks as a true multilinear fold of the already-materialized lincheck stripe at the pinned challenges, undoing the over-folded eq factors with a closed-form constant.

Mechanism: Removes the per-window bit transpose entirely; the drain's gathers per (lane, b) drop from three to one.

Result: Round 1 1277.5 → 1204.7 ms (−5.7%), 8/8; added-fold cost ≈ 180 ms nets against ≈ 250 ms of removals.

In the fast hash setups $C = I$, so $\hat{c} = \hat{z}$ and everything round 1 emits on the C side is *linear* in the witness. Concretely the two banks are, for parity $p \in \{0, 1\}$ and lane λ,

$$\mathrm{bank}_p[\lambda] \;=\; \sum_{K \equiv p \ (2)} \alpha^K \sum_b \frac{\gamma^b}{D} \sum_t \mathrm{eq}(r_{\geq 13}, t)\; z[\lambda, K, b, t], \tag{2}$$

a multilinear evaluation — which the baseline nevertheless computed with the quadratic side's machinery: a bit transpose of every 8192-bit window plus 32 of the drain's 48 gathers per lane.

By (1), a *true* multilinear fold at the actual pinned challenges reproduces the weights in (2) exactly. The prover already materializes the *lincheck stripe*, a repacking of z indexed $[\, i_{\text{inner}} \,|\, i_{\text{outer}} \,]$, for the later lincheck phase. So:

1: $v \leftarrow \mathrm{fold\_stripe\_outer}(z_{\text{stripe}},\ \mathrm{eq}(r_{k_{\log}..m}))$ ▷ the one $O(2^m)$ pass; same kernel lincheck uses
2: **for** $d = k_{\log} - 1$ **downto** 7 **do** ▷ data halves each round: $2^{14} \to 2^7$ elements
3: $\quad v[i] \leftarrow v[i] + r_d\,(v[i] + v[i + 2^d])$
4: **end for**
5: $\mathrm{bank}_p[\lambda] \leftarrow \dfrac{\mathrm{eq}(r_6, p)}{C_s}\; v[\, p \cdot 64 + \lambda \,]$ ▷ $C_s = (1{+}r_6)(1{+}r_7)(1{+}r_8)$, from r itself

Dimension 6 (the parity) and dimensions 0..5 (the lanes) are kept unfolded; the constant in step 3 undoes the two eq factors the partial fold applied, derived from the identity $Y_p = (C_s/\mathrm{eq}(r_6, p))\,\mathrm{bank}_p$ obtained by factoring (1) over the folded dimensions. With the banks gone from the drain, the workers run AB-only: the per-window bit transpose is deleted and the drain issues one gather per (lane, b) instead of three. Equivalence was proven at two levels before wiring: banks bit-identical to the drain's, and all three entry outputs identical, dense and padded.

Measured: round 1 1277.5 → 1204.7 *ms (−5.7%),* 8/8. *The cost of the added fold (≈ 180 ms) is included; the removals (≈ 250 ms) net out.*

2.3 Instruction-level parallelism in the gather drain

Baseline: One lane per iteration exposes three serial dependency chains (one per bank) to the out-of-order engine.

Improvement: Process two lanes per iteration so twice as many independent chains are in flight, with no change in total work.

Mechanism: 3 dependency chains in flight → 6 dependency chains in flight per iteration (same gather count and table size).

Result: Round 1 −63.1 ms (−4.3%), 8/8; combined with §2.4: 1477.3 → 1403.1 ms (−5.0%).

The drain accumulates, per output lane, three XOR chains of depth $n_{\text{med}} = 16$ (one per bank), each link a table gather feeding an XOR. The gathers are independent but the chains are serial, so one lane exposes three dependency chains to the out-of-order engine. Processing two lanes per iteration doubles that to six with no change in total work:

```
for ℓ = 0, 2, 4, ..., 62 do
    u0, u1, u2 ← 0;   v0, v1, v2 ← 0          ▷ banks for lanes ℓ and ℓ+1
    for b = 0 ... n_med − 1 do
        u_j ⊕= convert[b][·];   v_j ⊕= convert[b][·]     ▷ 6 independent chains in flight
    end for
end for
```

This targets latency, not throughput: an attempt to shrink the gather *table* (64 → 8 KiB, doubling gather count) had cost 3.6% — the table already fit M1's 128 KiB L1D, establishing that the drain is bound by gather count and chain depth, not footprint.

Measured: round 1 −63.1 ms (−4.3%), 8/8. *Combined with Section 2.4:* 1477.3 → 1403.1 *ms (−5.0%).*

2.4 Structurally fixed rows of the b operand

Baseline: Every K-row of b is pushed through the $\mathbb{F}_2$-linear table T, including the 15 of 256 rows the circuit pins to 0^8 regardless of input.

Improvement: Guard each row with a single zero check; since $T(0) = 0$, a zero b-row contributes nothing and its work can be skipped outright.

Mechanism: Skips 64 table loads and 4 $\mathbb{F}_{2^8}$ multiplies for each of the 15 structurally-zero K-rows (out of 256).

Result: Round 1 −42.4 ms (−2.9%), 8/8.

Round 1's dominant kernel transforms the operands a, b through a table T implementing the inverse-NTT composition; T is $\mathbb{F}_2$-linear, so $T(0) = 0$. A census of the packed BLAKE3 witness (256 word positions per block × 256 blocks × 3 independent witnesses) shows the circuit pins 38 of 256 eight-byte K-rows of b regardless of the inputs, taking only three values:

$$\underbrace{\texttt{0xff}^8}_{\text{22 positions (const-one wires)}}, \qquad \underbrace{0^8}_{\text{15 positions (structural zeros)}}, \qquad \text{one mixed word.}$$

For a zero row the entire row's work vanishes: $db = T(0) = 0$, hence $y = da \odot db = 0$ contributes nothing to any accumulator. The kernel gains a single guard:

```
if load64(b_row) = 0 then return      ▷ skips all 64 table loads and 4 F_{2^8} multiplies
```

No census data ships and no positions are tracked: the guard is exact for any witness, and a disagreeing witness falls through to the generic path. (The all-ones rows were also tried: constant-folding them saves only the b half of a row's work, and halving a row's loads halves its memory-level parallelism, returning 6 ms where 27 were predicted. Reverted; whole-row elimination is what pays.)

Measured: round 1 −42.4 ms (−2.9%), 8/8.

3 Zerocheck round 2

Two kept changes, sharing the deferred-reduction machinery, took round 2 from 433 to 312 ms (−28%).

3.1 Deferred reduction in vector registers

Baseline: The 256-bit accumulator lived as a four-word struct in general registers: six lane extracts per product to leave the vector file, four scalar XORs per accumulate.

Improvement: Keep the accumulator resident as two 128-bit vector registers and form the unreduced product with Karatsuba.

Mechanism: Karatsuba product: three `PMULL`s instead of the schoolbook four; accumulate: four scalar XORs → two vector XORs; the six lane extracts move from per-product to once per worker chunk.

Result: Round-2 phase 433.1 → 375.2 ms (−13.4%).

For $u \in \mathbb{F}_2^{256}$ (an unreduced carry-less product) let $R(u) \in \mathbb{F}_{2^{128}}$ be reduction mod $X^{128} + X^7 + X^2 + X + 1$. R is $\mathbb{F}_2$-linear, so for any products $u_i = a_i \odot b_i$ (unreduced),

$$\textstyle\sum_i R(u_i) \;=\; R(\sum_i u_i)\,,$$

and a sum of n field products needs one reduction, not n. The baseline already used this identity, but held the 256-bit accumulator as a four-word struct in *general* registers: each product paid six lane extracts to leave the vector file, and each accumulate was four scalar XORs.

The fix is representational, not algorithmic: keep the accumulator as two 128-bit vector registers. The unreduced product uses Karatsuba,

$$(a_l + a_h X^{64})(b_l + b_h X^{64}) : \quad ll = a_l b_l, \quad hh = a_h b_h, \quad cross = (a_l{+}a_h)(b_l{+}b_h) + ll + hh,$$

three `PMULL`s instead of the schoolbook four; accumulation is two vector XORs; the four extracts are paid once per worker chunk on the way out, and the existing scalar reducer is reused unchanged.

Measured: round-2 phase 433.1 → 375.2 *ms (*−13.4%*).*

3.2 The register-resident message loop

Baseline: The message loop crossed the vector/general register-file boundary three times per output pair (fold-accumulator extract, scalar field multiply in/out, re-entry for accumulation), at six `PMULL`s per pair.

Improvement: Keep the whole chain — fold, in-register Karatsuba multiply, and accumulate — resident in vector registers.

Mechanism: 6 `PMULL` → 5 `PMULL` per pair (three for the product, two for the fold through $X^{128} \equiv X^7{+}X^2{+}X{+}1$); all three per-pair register-file crossings removed, leaving only four stores and one eq load.

Result: Round-2 phase 358.1 → 311.6 ms (−13.0%), 8/8, every pair in [−12.6%, −13.3%].

Round 2's message loop crossed the vector/general register-file boundary three times per output pair: the table-driven fold extracted its vector accumulator into a two-word struct; the message products $g_1 = a_1 b_1$, $g_\infty = (a_0{+}a_1)(b_0{+}b_1)$ went through the scalar field multiply (struct in, struct out); and the accumulate of Section 3.1 re-entered the vector file. The rewrite keeps the whole chain in vector registers: the fold returns its register unextracted; the products use an in-register Karatsuba multiply (five `PMULL`s — three for the product, two for the fold through $X^{128} \equiv X^7{+}X^2{+}X{+}1$ — against the baseline's six); the $\mathrm{eq} \cdot g$ products feed the Section 3.1 accumulator directly. Per pair, the only remaining memory traffic is the four required stores of folded values and one eq load.

Measured: round-2 phase 358.1 → 311.6 *ms (*−13.0%*),* 8/8*, every pair in* [−12.6%, −13.3%]*, taken on a machine with an active browser session (the min-of-5 protocol absorbs interactive bursts).*

4 Zerocheck rounds 3+

One structural change took the tail from 455 to 307 ms (−33%).

4.1 Fusing away a memory pass

Baseline: Two passes per worker chunk: fold and store a, b, then re-read the just-written multi-megabyte chunk from L2/DRAM to build the message.

Improvement: Fold each output pair in-register, store once, and consume the pair for the message while still in registers — no re-read.

Mechanism: Memory traversals per worker chunk: 2 → 1; `PMULL` count is unchanged from the two-pass form.

Result: Rounds-3+ phase 449.5 → 306.6 ms (−31.8%), 8/8, every pair in [−31.1%, −32.6%] — the largest single win of the effort.

The tail rounds apply the register-resident treatment of §3.2 plus one thing round 2 had no opportunity for. The incumbent made *two* passes per worker chunk: fold $a_{\text{in}}, b_{\text{in}}$ into $a_{\text{out}}, b_{\text{out}}$, then re-read the just-written multi-megabyte chunk to build the message — an L2/DRAM read of data that had been in vector registers moments earlier. The fused kernel folds each output pair in-register ($e + r\,(e + o)$ via `mul_q`), issues the one required store, and consumes the pair for the message while still in registers:

1: **for** $x = 0 \ldots lo - 1$ **do**
2: $\quad a_0, a_1, b_0, b_1 \leftarrow$ fold four input pairs at r ▷ in q registers
3: $\quad$ store a_0, a_1, b_0, b_1 ▷ the required write, once
4: $\quad g_1 \leftarrow a_1 b_1$; $\quad g_\infty \leftarrow (a_0 + a_1)(b_0 + b_1)$ ▷ `mul_q`, never leaving registers
5: $\quad acc_1 \oplus\!= \mathrm{eq}_x \odot g_1$; $\quad acc_\infty \oplus\!= \mathrm{eq}_x \odot g_\infty$ ▷ unreduced, §3.1
6: **end for**

The `PMULL` count is identical to the two-pass form; the entire gain is one traversal instead of two plus the removed struct crossings. Notably, two earlier attempts on this exact loop had failed (a memory-interfaced pair-multiply kernel, +10%; wide accumulators alone, 0%): they changed the arithmetic encoding and the accumulator while leaving the pass structure intact, which is where the cost actually lived.

Measured: rounds-3+ phase 449.5 → 306.6 *ms (*−31.8%*),* 8/8*, every pair in* [−31.1%, −32.6%] *— the largest single win of the effort.*

5 Lincheck

5.1 One word load per stripe, two stripes per pass

Baseline: 16 loads per stripe step: 8 table entries plus 8 separate scalar byte loads for the indices.

Improvement: Load the 8 adjacent index bytes as one u64 with GPR shift-extracts, and fold two stripes per iteration so the table-entry XOR fuses to a single `EOR3`.

Mechanism: 16 loads/stripe → ≈9 loads/stripe.

Result: `partial_fold_z` ST 40.7–40.8 → 27.8–28.0 ms (−32%, 3/3 disjoint); 8T −28%.

The lincheck's dominant pass folds the 128 MB packed witness through per-stripe 256-entry byte tables into eight Q-register accumulators. The loop is load-port bound (M1: 3 loads/cycle), and the incumbent spent 16 loads per stripe step: 8 table entries *plus 8 scalar byte loads* for the indices. The indices are 8 adjacent bytes — one u64 load and GPR shift-extracts replace all eight — and folding two stripes per iteration lets the two table entries enter the accumulator as an XOR pair, which LLVM fuses to a single `EOR3` under the SHA3 feature: ≈9 loads per stripe, same XOR multiset, bit-identical. Measured, paired: `partial_fold_z` ST 40.7–40.8 → 27.8–28.0 ms (−32%, 3/3 disjoint) — landing on the pass's computed ≈28 ms load floor; 8T −28%. This supersedes the earlier "no reachable headroom" verdict, which had priced blocking and table-geometry changes but not the index-load restructure (idea from the challenge tree's asm kernel, re-expressed in intrinsics).

6 PCS open

6.1 Composing the block scalar into the fold table

Baseline: The combine pass spent one field multiply plus one 16-load table fold per slot per claim — 58% of open time.

Improvement: Fold the block-constant multiply into a freshly composed per-(claim, block) table, built once from 127 shift-and-fold doublings plus subset-sum expansion, so the sweep only indexes the composed table.

Mechanism: Removes $2L \approx 16.8$M per-slot field multiplies per open, replaced by a one-time ≈4.3k-word-op build per block (127 shift-and-fold doublings plus $16 \cdot 255$ subset-sum additions).

Result: Combine 94.4 → 78.5 ms, open total ≈ 160 →≈ 145 ms (−9%); at 8 threads combine is 10.9 ms, a near-linear transfer.

Shape for this section: the 2^{16}-compression open ($m = 30$, $L = 2^{23}$ packed slots, two batched claims). The ring-switch reduction leaves, for each claim k, an eq tensor in factored form $\mathrm{eq} = \mathrm{eq}_{\mathrm{lo}} \otimes \mathrm{eq}_{\mathrm{hi}}$ (lengths B and L/B) and an $\mathbb{F}_2$-linear map $L_k : \mathbb{F}_{2^{128}} \to \mathbb{F}_{2^{128}}$ (the γ_k-weighted byte-table fold). The *combine* pass materializes the batched basis

$$b[j] \;=\; \sum_k L_k\big(\mathrm{eq}_{\mathrm{lo}}[j \bmod B] \cdot \mathrm{eq}_{\mathrm{hi}}[\lfloor j/B \rfloor]\big), \qquad j < L,$$

and was the largest bucket of the open: one field multiply plus one 16-load table fold per slot per claim, 58% of open time.

The multiply is removable. Multiplication by the block constant $e = \mathrm{eq}_{\mathrm{hi}}[\lfloor j/B \rfloor]$ is itself $\mathbb{F}_2$-linear, so the per-slot map $G_{k,e}(w) = L_k(w \cdot e)$ is a composition of $\mathbb{F}_2$-linear maps and is fully determined by its 128 monomial images $g_r = L_k(X^r e)$. Once per (claim, block), encode $G_{k,e}$ as a fresh 16×256 byte table: advance $X^r e$ by 127 exact shift-and-fold doublings (multiplication by X costs one shift and a conditional XOR of the reduction word), evaluate L_k on each via the existing table, and expand every byte position from its 8 generators by subset-sum doubling ($16 \cdot 255$ additions). About 4.3k word operations per block; the sweep then indexes only the composed table, and the per-slot multiply — $2L \approx 16.8$M multiplies per open — is gone.

The catch is the block length. The balanced split used by the many-pass folds ($B = 2^{\lceil n/2 \rceil} = 2^{11}$ here) is too fine for the build to amortize; the deferred claims carry their own coarse split $B = \min(2^{15}, 2^{n-8})$, making $\mathrm{eq}_{\mathrm{lo}}$ a single 512 KiB L2-resident stream shared by every block and the build ≈ 0.4 ops/slot. Correctness is exact, not approximate: every composed entry is an XOR combination of exact field products — the same XOR multiset as the slot-multiply path — so the transcript is bit-identical (unit equivalence test; verification of full proofs unchanged).

Measured (ST, minimum over samples): combine 94.4 → 78.5 ms, open total ≈ 160 →≈ 145 ms (−9%); at 8 threads the combine is 10.9 ms, a near-linear transfer. The idea was found dormant in the challenge tree (present, never isolated or measured there) and re-derived.

Negative space for the open. (i) Three-way XOR fusion (`EOR3`) in the fold's reduction tree is flat: LLVM already emits it under `target-cpu=native`, and the sweep is bound by its 32 loads per slot, not its XOR count. (ii) Fusing both claims into one sweep — two live composed tables, one store, no read-back of the intermediate — *regresses* 15%: 128 KiB of gather targets thrash L1, so the claim-sequential order stands. (iii) The round-1 lookahead coefficients accumulated in the combine's tail are an accounting wash, not a speedup: tail-plus-initial-sumcheck costs 60.5 vs. 60.6 ms across the two trees' opposite placements of the same work.

7 Merkle hashing: BLAKE3

7.1 Two transposed four-wide states in flight

Baseline: The crate's NEON backend runs a single 4-wide transposed state (one `uint32x4` per state word); BLAKE3's serial G function leaves Apple's four NEON pipes half idle.

Improvement: Interleave a second independent transposed 4-wide state, G-for-G, to fill the idle pipe slots.

Mechanism: 1 transposed 4-wide state (4 messages in lockstep) → 2 interleaved 4-wide states (8 messages in flight, the full 32-register NEON file); no added work per message.

Result: ST tree build 1.63 → 2.30 GB/s at 512 B leaves (+41%) and 1.71 → 2.42 GB/s at 1 KiB leaves (+42%).

The PCS Merkle tree can hash with SHA-256 (hardware silicon, ≈2.5 GB/s per core) or BLAKE3. The blake3 crate's NEON backend processes four messages in lockstep — one `uint32x4` per state word — but a single 4-wide state is *latency*-bound: BLAKE3's G function is a serial add–xor–rotate chain, and Apple's four NEON pipes sit half idle waiting on it. Interleaving a *second* independent transposed 4-wide state, G-for-G (eight messages in flight, the full 32-register NEON file), fills the pipes — the same independent-chains pattern as §2.3. Dispatch: groups of eight leaves or parent pairs take the new kernel; tails fall to the crate path; byte-identical by an equivalence test over all batched message sizes, counts, and flag combinations.

Measured (ST tree build): 1.63 → 2.30 GB/s at 512 B leaves (+41%, now 1.08× *faster* than SHA-256) and 1.71 → 2.42 GB/s at the 1 KiB production-geometry leaves (+42%, parity with the SHA silicon). The challenge tree reaches ≈2.58 GB/s with a 2.6k-line generated assembly kernel; this is ≈290 lines of intrinsics within 6% of it — LLVM absorbed the full register pressure without measurable spill cost. End-to-end the change erases the −5% penalty BLAKE3-Merkle previously carried in this tree, making the Merkle hash choice free.

8 Multithreading

The campaign target mirrors the single-threaded closure: CPU-only multithreaded throughput within 10% of the challenge tree. Start: 1.19× apart at the 2^{16} shape. Standing after the two subsections below: 1.146× (149.3 vs 130.2 ms, paired same-minute, production configuration). All numbers here are 8-thread-class production runs at 2^{16} compressions.

8.1 Round-1 AB prep under the commit, on all cores

Baseline: Overlapping the witness-only AB transform with the commit under `rayon::join` measured a wash twice on the P-core-only pool; the transform's output buffer was a fresh zeroed allocation.

Improvement: Draw the output buffer from the scratch pool uninitialized, and run the join window on the all-core (P+E) pool instead of P-cores only.

Mechanism: Not an operation-count change but a memory-traffic and scheduling fix: removes the memset-plus-page-fault cost of a fresh zeroed allocation, and moves the overlap window onto the otherwise-idle E-core issue capacity left by the bandwidth-bound commit NTT.

Result: Kill-switch A/B 3/3 at 2^{16} (149.6–151.3 vs 152.5–156.3 ms), 2/2 at the ranked shape (clean pair −57 ms).

Round 1's dominant half — the shift-reduce AB transform — reads only the witness, so it can run before the commitment root exists. Overlapping it with the commit under `rayon::join` was measured a wash TWICE on the 8-P-core pool: both arms time-slice a saturated pool. Two changes flip it decisive: (i) the transform's 2^{m-13} KiB output buffer comes from the scratch pool *uninitialized* (a fresh zeroed allocation's memset + page faults consumed the entire overlap gain; slots for padding-skipped rows stay stale-but-unread, bounded by the same padding table every consumer uses), and (ii) the join window runs on an all-core (P+E) pool while the rest of the prove stays on P-cores. The efficiency cores are the active ingredient: the commit's NTT is bandwidth-bound and leaves issue capacity, and the prep is gather/PMULL compute the E-cluster can add without competing for DRAM. (The converse placement — a bandwidth-bound task like the lincheck stripe transpose on the E-cores during the commit — measured *negative*.) Paired kill-switch A/B: 3/3 at 2^{16} (149.6–151.3 vs 152.5–156.3 ms), 2/2 at the ranked shape (clean pair −57 ms). Bit-identical: the precomputed and inline transforms are equality-tested on all outputs.

8.2 Witness constant-region elision via scratch provenance

Baseline: The full-write witness builder rewrites every byte of z/a/b each prove, including regions — b's all-ones prefix, reserved-slot words, all three streams' zero tails — identical across every prove of a layout.

Improvement: Tag a pooled buffer with its provenance (derived independently at give and take sites) and, on a tagged hit, skip the constant-region writes.

Mechanism: No explicit byte or operation count is given in this section; this is a work-removal via provenance tracking, not a stated op-count reduction.

Result: Paired kill-switch A/B: 4/5 (147.0–149.0 vs 148.3–152.7 ms).

The full-write witness builder rewrites every byte of z/a/b each prove — including b's all-ones prefix and reserved-slot words and all three streams' zero tails, which are identical for every

prove of a layout. A provenance tag (derived independently at the buffer's give site from the R1CS constants and at its take site from the encoder's own constants — no plumbing carries it) marks a pooled buffer as holding exactly a previous prove's output of this layout; any other pool custody clears the tag. On a tagged hit the builder skips the constant-region writes. Idea from the challenge tree's scratch tokens, re-derived; paired kill-switch A/B: 4/5 (147.0–149.0 vs 148.3–152.7 ms). Byte-identity is enforced by a test that drives a tagged give/take cycle against a fresh generation.

Negative space for multithreading. Seven mechanisms measured null or inverted on this machine, all paired: the P-pool-only join (twice); non-temporal stores in the witness writers (M1 ignores the hint — third confirmation); the stripe transpose on E-cores during the commit (−, bandwidth-on-bandwidth); the all-core PCS combine; four-layer NTT fusion on aarch64 (+19–26%, sixteen live F128s spill); a two-block scalar witness interleave (+40% ST, GPR blowout); a 4-wide SIMD witness builder with scalar packing (the packing, not the hash math, is the cost); and the full lane-wise vectorized packing network itself (§1.2 — null once the stripe-buffer allocation was fixed). What separates the remaining few percent at the production shape: the challenge tree's compact round-2 anchor+delta format with symbolic round-collapsing lookaheads — a thousand-line-class port, priced and declined at the current bloat bar pending an explicit call.

Summary

Section	Kind	Phase effect (ST)	Sign test
§3.1	representation	round 2: −13.4%	decisive
§2.4	work removal	round 1: −2.9%	8/8
§2.3	ILP	round 1: −4.3%	8/8
§2.1	arithmetic	round 1: −9.1%	8/8
§2.2	structural	round 1: −5.7%	8/8
§3.2	representation	round 2: −13.0%	8/8
§4.1	pass fusion	rounds 3+: −31.8%	8/8
§6.1	work removal	open combine: −17%	decisive
§1.1	work removal	witness gen: −24%	3/3
§5.1	work removal	lincheck fold: −32%	3/3
§7.1	ILP	BLAKE3 merkle: +41% GB/s	decisive
round 1 cumulative		1477 → 1205 ms	(−18.4%)
round 2 cumulative (§3.1+§3.2)		433 → 312 ms	(−28%)
rounds 3+ cumulative		455 → 307 ms	(−33%)
PCS open cumulative (§6.1)		≈ 160 →≈ 145 ms	(−9%)
end-to-end, all seven zerocheck changes		≈ 52,400 →≈ 58,100 comp/s	(≈ +11%, 8/8)

Negative space. The kept changes are one side of a 17-experiment record; the failures shaped them. On this core, re-encoding fixed work never paid: pairing multiplies through a memory-interfaced kernel (+10%), a four-register structured load (+12%, microcoded), three-way XOR fusion (+1.8%), an 8× smaller gather table (−3.6% regression), and deferring an already-cheap reduction (0%) all failed, while every win either removed work, added independent work in

flight, or removed register-file boundary crossings. In the commit-phase NTT, rewriting the butterfly multiply as a 3-PMULL Karatsuba with a q-resident interface regressed 58%: the generic butterfly already inlines the latency-optimal binius kernel, and six PMULLs with no repack beat three with one, in situ and not only in the microbenchmark. Two further structural transplants measured neutral end-to-end and were reverted despite making the zerocheck *bucket* look better (hoisting the challenge-independent prep under the commit; the lookahead double-round, which loses 7–15% in the zerocheck tail on both this machine and the reference tree's, while *helping* the PCS open by 7% — the same technique, opposite signs, two call sites).